

Introdução

Programação Funcional

Marco A L Barbosa

malbarbo.pro.br

Departamento de Informática

Universidade Estadual de Maringá



Este trabalho está licenciado com uma Licença Creative Commons - Atribuição-Compartilhagual 4.0 Internacional.

<http://github.com/malbarbo/na-progfun>

O que é?

O que é programação imperativa?

- Um paradigma de programação em que os programas são descritos com sentenças que modificam o estado do programa.

O que é programação funcional?

- Um paradigma de programação em que os programas são descritos com aplicação e composição de funções.
- Evita mudança de estado (mudança do valor das variáveis)
- Evita efeitos colaterais (qualquer efeito que seja observável além do valor de saída da função, como a mudança dos parâmetros ou de variáveis globais, exceções, entrada e saída, etc.)

Por quê?

Um paradigma (linguagem) de programação é uma ferramenta.

Conhecer várias ferramentas permite utilizar a mais adequada para cada problema.

Compartilhamento de dados com mudança de estado é difícil!

Qual o valor de `lst`?

```
>>> lst = [0] * 3
>>> lst
[0, 0, 0]
>>> lst[1] = 10
>>> lst

[0, 10, 0]
```

Qual o valor de `lst`?

```
>>> lst = [[]] * 3
>>> lst
[[], [], []]
>>> lst[1].append(2)
>>> lst

[[2], [2], [2]]
```

```
def adiciona_todos(
    dest: list[int],
    fonte: list[int]):
    ...

    Adiciona todos os elementos
    de *fonte* no final
    de *dest*.
    ...

    for x in fonte:
        dest.append(x)
```

Qual o valor de `lst`?

```
>>> lst = [4, 3, 1]
>>> adiciona_todos(lst, [6, 2])
>>> lst

[4, 3, 1, 6, 2]

>>> adiciona_todos(lst, lst)
>>> lst
```

A execução não para!

Nas linguagens puramente funcionais, como o Gleam, não existe mudança de estado: depois de criado, um valor nunca muda.

Sem mudança de estado, os problemas que acabamos de ver não existem.

As duas definições a seguir são equivalentes?

```
def soma_indices(lst: list[int], a: int, b: int) -> int:  
    return indice(lst, b) + indice(lst, a)
```

```
def soma_indices(lst: list[int], a: int, b: int) -> int:  
    return indice(lst, a) + indice(lst, b)
```

```
# indice(lst, i) devolve o elemento de índice i de lst
```

Não é possível afirmar que as duas definições são equivalentes sem olhar o código da função `indice`. Se a função `indice` tem efeitos colaterais, então as definições podem não ser equivalentes.

Mas nós não confiamos no propósito das funções?

Sim, o propósito diz o que a função produz, mas nem sempre tudo que ela faz. Com o tempo, um “jeitinho” para corrigir um problema rapidamente, como criar uma variável global, pode fazer o corpo divergir do propósito.

A possibilidade de efeitos colaterais **dificulta pensar localmente** sobre o funcionamento do código.

A ausência de efeitos colaterais **permite pensar localmente** sobre o funcionamento do código.

Como?

1) Escolher uma linguagem

- Gleam: simples, com bom suporte ao paradigma funcional e inferência de tipos
- Vamos usar o `sgleam` (*Student Gleam*), um interpretador de Gleam voltado para o ensino e de instalação simples, assim como usamos o `Spython` para o Python

2) Estudar as construções do paradigma e as referências da linguagem

3) Praticar lendo e escrevendo código, com muitos exemplos e muitos exercícios

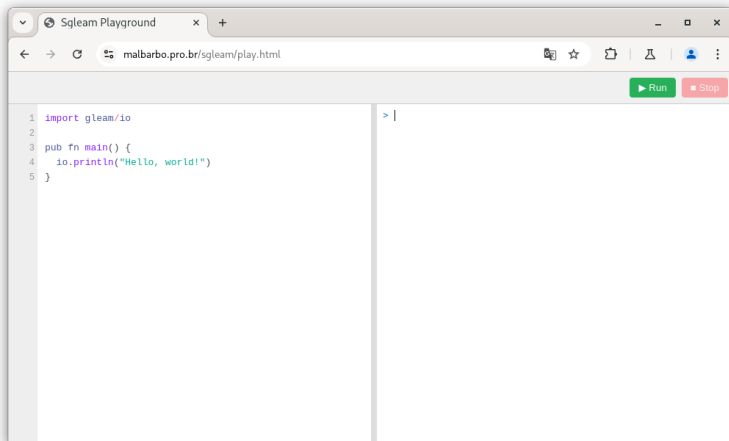
2) Estudar as construções do paradigma e as referências da linguagem

- [A Tutorial Introduction to the Lambda Calculus](#)
- Livro [How to Design Programs](#)
- Livro [Structure and Interpretation of Computer Programs](#)
- [Tour](#) da linguagem Gleam
- [Documentação](#) da biblioteca padrão da linguagem Gleam
- [Página](#) do sglean

Primeiros passos

Para programar em Gleam sem instalar nada, acesse <https://malbarbo.pro.br/sgleam/play.html>.

O programa é escrito à esquerda e o botão **Run** o executa, exibindo o resultado à direita.



No Linux

```
$ curl -s -L https://malbarbo.pro.br/sgleam/sgleam.tar.gz | tar xvz
```

ou

```
$ wget -qO- https://malbarbo.pro.br/sgleam/sgleam.tar.gz | tar xvz
```

Em outros sistemas

Acesse <https://malbarbo.pro.br/sgleam/> e faça o download e a descompactação manualmente.

Considere o arquivo `dobro.gleam` com o conteúdo

```
pub fn dobro(x: Int) -> Int {  
  x * 2  
}
```

```
pub fn smain() {  
  dobro(4)  
}
```

O `sgleam` avalia a chamada `smain()` e exibe o valor produzido.

Para executar o arquivo (função `smain`) no Linux digite

```
$ ./sgleam dobro.gleam  
8
```

No Windows

```
PS> .\sgleam dobro.gleam  
8
```

No REPL (*Read Eval Print Loop*) cada expressão digitada é lida, avaliada e tem o resultado exibido, e o processo se repete, como na janela de interações do Spython. Vamos usar os dois nomes, REPL e **janela de interações**, para nos referir a esse ambiente.

Para iniciar o REPL

```
$ ./sgleam
```

```
Welcome to sgleam.
```

```
Type ctrl-d or ":quit" to exit.
```

```
> 2 + 5
```

```
7
```

Para carregar um arquivo e iniciar o REPL

```
$ ./sgleam repl dobro.gleam
```

```
Welcome to sgleam.
```

```
Type ctrl-d or ":quit" to exit.
```

```
> dobro(4)
```

```
8
```

Revisão

O que é programação imperativa?

- Um paradigma de programação em que os programas são descritos com sentenças que modificam o estado do programa.

O que é programação funcional?

- Um paradigma de programação em que os programas são descritos com aplicação e composição de funções, evitando mudança de estado e efeitos colaterais.

Se já sabemos programar, por que estudar outro paradigma?

- Um paradigma é uma ferramenta. Conhecer várias ferramentas permite utilizar a mais adequada para cada problema.

O que é um efeito colateral?

- Qualquer efeito de uma função que seja observável além do seu valor de saída, como a mudança dos parâmetros ou de variáveis globais, exceções, entrada e saída, etc.

Por que a possibilidade de efeitos colaterais é um problema?

- Ela dificulta pensar localmente sobre o código: não é possível entender o que uma função faz sem olhar o código das funções que ela chama.

Por que compartilhar dados com mudança de estado é difícil?

- Quando dois trechos de código compartilham um valor, a mudança feita por um é observada pelo outro, muitas vezes de forma inesperada.

Como as linguagens puramente funcionais evitam esse problema?

- Elas não têm mudança de estado: depois de criado, um valor nunca muda, então nenhum trecho de código pode alterar o valor que o outro está usando.

Que linguagem vamos utilizar e por quê?

- Gleam, porque é simples, tem bom suporte ao paradigma funcional e inferência de tipos. Vamos utilizá-la através do `sgleam`, um interpretador voltado para o ensino.

O que é um REPL?

- Um ambiente interativo que lê uma expressão (*Read*), avalia a expressão (*Eval*), exibe o resultado (*Print*) e repete o processo (*Loop*). Também chamamos esse ambiente de janela de interações.

Referências

Básicas

- [Tour da linguagem Gleam](#)
- [Programação funcional](#)

Complementares

- [The Python paradox](#)
- [Revenge of the Nerds](#)
- [Beating the averages](#)